

RECON

Workshop Curriculum — Reconnaissance Report

Purpose. Input for a SpecKit `specify` phase. This document records **what already exists** in this repository so the specification is built on measured reality rather than assumption. It contains no design and no recommendations that are not clearly labelled as such.

Date of survey: 2026-08-31 **Repository root:** `<ext-volume-host>/Projects/vasic` **Repository HEAD at survey time:** 708c8b1 (branch `main`) **Mode:** read-only reconnaissance. Nothing outside this file was created or modified.

§0 · How to read this document, and what “verified” means

Per Constitution §11.4.6, every statement below is one of three kinds, and they are never mixed:

Marker	Meaning
<i>(unmarked)</i>	Measured. Produced by a command listed in §7 Command log . The command that produced it is named inline or in §7.
UNVERIFIED	Could not be established in this session. The reason is always given.
<i>(labelled “prose only”)</i>	Present in this tree as documentation , with no backing implementation here .

Marker	Meaning
	This distinction is load-bearing for §3.

Three constraints applied to this survey and shaped what could be measured:

1. A lumen index --force rebuild (PID 897646) and a constitution gate sweep were running. No indexing, no ollama restart, no heavy build was performed.
2. Another agent was concurrently editing helix-deps.yaml, scripts/verify-governance-cascade.sh and scripts/setup-agents-wizard.sh, and adding governance carriers to workshop/. Those files were read, never written.
3. **The ai_interviewing platform was not started.** Its code, schema, scripts and test banks were read in full; its *runtime* behaviour is therefore **UNVERIFIED by this survey**. curl http://localhost:8099/api/health returned nothing (connection refused); platform/run/server.json is a stale artefact naming pid 49211, which is not alive. Every runtime claim below is attributed to the code or to platform/README.md, never asserted as observed.

§1 • workshop/ — exhaustive inventory

workshop/ is a git submodule declared in .gitmodules:

```
[submodule "workshop"]
  path = workshop
  url = git@github.com:milos85vasic/workshop_curriculum.git
```

Gitlink at superproject HEAD:

ff59fca320efd839c9322bcf6f683ccb74754da8 (heads/main, working tree clean). workshop/.git is a gitfile pointing at ../.git/modules/workshop.

1.1 Directory tree (complete — every path)

```
workshop/
├── .gitignore
├── README.md
├── chapters/
│   └── 01/
│       ├── Milos teaching ... AI workflows - 2026_08_27 09_57 CEST
│       ├── Notes by Gemini.PDF
│       └── Milos teaching ... AI workflows - 2026_08_27 09_57 CEST
```

```

- Recording.mp4
|   └─ Milos teaching ... AI workflows - 2026_08_27 09_57 CEST
- Recording.mp4.sha256
|   └─ Milos teaching ... AI workflows - 2026_08_27 09_57 CEST
- Recording.mp4.tar.gz.part-000 ... part-035
└─ docs/
   └─ superpowers/
      └─ specs/
         └─ 2026-08-28-video-archive-pipeline-design.md
└─ scripts/
   └─ archive-videos.sh
   └─ extract-videos.sh
   └─ install-hooks.sh
   └─ self-test.sh
   └─ git-hooks/
      └─ post-checkout
      └─ post-commit
└─ upstreams/
   └─ GitHub.sh

```

Redaction. The ... inside the four chapters/01/ filenames is a **deliberate elision of a third party’s first name**, not a truncated path. It is the only character removed from those four names; everything else — the subject, the date, the time, the zone and every extension — is verbatim. The recording is addressable without it: workshop/chapters/01/ holds exactly one *Recording.mp4, and a passage inside it is addressed by chapter slug plus pid. See [docs/content-boundary-incident-2026-09-01.md](#) §11.

There are **no** governance carriers (CLAUDE.md / AGENTS.md / QWEN.md / GEMINI.md), **no** helix-deps.yaml, **no** Containerfile/Dockerfile/compose file, **no** source code, **no** curriculum content of any kind. This state is **in flux** — another agent was adding carriers during this survey (CONTINUATION.md records G7 as “20 of 24 ... workshop/ onboarding in flight”).

1.2 File counts and sizes (counted items, not line counts)

Location	Items	Bytes on disk	Notes
workshop/ total working tree	—	3.5 G (du -sh)	includes the extracted .mp4
.git/ modules/ workshop	—	1.8 G	history holds the 36

Location	Items	Bytes on disk	Notes
			archive parts
Files tracked by git (git ls-files)	48	—	the .mp4 is not among them
chapters/	1 dir (01/), 39 files	3.5 G	—
chapters/01/ — video (git-ignored)	1	1,871,981,557	*.mp4 is in .gitignore
chapters/01/ — archive parts (tracked)	36	1,870,454,585	35 × 52,428,800 + 1 × 35,446,585
chapters/01/ — manifest (tracked)	1	5,811	*.mp4.sha256
chapters/01/ — PDF (tracked)	1	417,800	Gemini meeting notes
docs/	1 file	4.0 K	one design spec
scripts/	6 files	28 K	4 .sh + 2 git hooks
upstreams/	1 file	4.0 K	GitHub.sh

1.3 What chapters/ actually contains

Exactly one chapter — 01. No 02. No manifest, index, ordering file, metadata file, or naming convention beyond the two-digit directory name.

The video (measured with ffprobe):

Property	Value
Container	mov,mp4,m4a,3gp,3g2,mj2, major_brand: isom, encoder: Google
Duration	01:55:28.75 (6,928.75 s)
Overall bitrate	2,161 kb/s
	h264 (High), yuv420p, 1920×1080 , 2,029 kb/s, 24 fps

Property	Value
Video stream	
Audio stream	AAC-LC, 48 000 Hz, stereo, 128 kb/s
Size	1,871,981,557 bytes
SHA-256 (from manifest)	345a74398a440e2df3a8cacf8422e7712c3e5bab19147594811b44787dbed03f

The PDF is Notes by Gemini — 8 pages, letter, produced 2026-08-27 19:33 CEST by Skia/PDF m150 from a Yandex-Browser-on-Android print. pdftotext -layout yields **3,044 words / 23,656 characters**.

This is a summary, not a transcript. Its structure is Gemini’s meeting-notes format: a Quick notes section of thematic bullets grouped under **six topic headings**, a Next steps action list attributed to named participants, and a Full notes section of paragraph-length narrative bullets. There are **no timestamps**, **no speaker-turn boundaries**, and **no verbatim utterances** anywhere in the file. A 3,044-word summary of a 115-minute conversation is roughly 26 words per minute; ordinary speech is 130–160 wpm. **No verbatim transcript of chapter 1 exists in this repository.**

The six heading strings are not reproduced here. They are private content; see the content boundary note below. Read them from the source if you need them.

Content signal relevant to the spec, **described rather than quoted** (see the content boundary note below). The notes identify two participants — the operator, and a third party whose full name is private content and is deliberately not written into this repository. They record a short list of assigned actions. In substance: the operator is the one who put forward the idea of turning the session recordings into an interactive curriculum that links references together and closes gaps in coverage; the intended end state is a canonical, maintainable body of knowledge with conversational access layered over it; and putting that material, together with the recordings, somewhere web-browsable is one of the assigned actions. The notes further confirm — again by description, not by quotation — that the tooling catalogued in §3 below is already in the

operator’s daily working set: semantic code indexing, the spec-driven planning extension, the operator’s own QA harness, and local llama.cpp inference fronted by an OpenAI/Anthropic-compatible proxy.

Content boundary — read before adding to this section

The source of everything in §1.3 is workshop/chapters/01/... - Notes by Gemini.PDF, inside the **private** workshop submodule. **This repository is public.** Three classes of material from that file must never appear here, in this document or any other: **verbatim text**, the **section-heading list**, and the **third party’s name**. Describe; do not reproduce. bash scripts/verify-content-boundary.sh enforces this. An earlier revision of this document violated all three — recorded, with the residual exposure, in docs/content-boundary-incident-2026-09-01.md.

1.4 scripts/ — what they actually do

None of these scripts build, run, serve, or containerize anything. All four exist solely to keep a multi-gigabyte video inside a git repository that rejects >100 MB blobs. Every one is set -euo pipefail and carries a documented header block (§11.4.18-shaped).

Script	What it does
archive-videos.sh [root] [--delete-source]	Finds *. {mp4,mov,mkv,webm,m4v,avi} under root, tar -czf - \ split -b \$PART_SIZE (default 50m) into a temp dir, hashes each part and the reassembled stream, verifies the stream with tar -tzf, writes a *.sha256 manifest, moves parts into place. Idempotent: re-runs print skip (already archived) when the video hash and every part hash still match.
extract-videos.sh [root]	For every *.sha256, verifies each part hash → the concatenated stream hash → extracts → verifies the extracted video hash. Three verification levels. Never deletes

Script	What it does
	archives. Idempotent (skip (intact)).
install-hooks.sh	Symlinks scripts/git-hooks/{post-commit,post-checkout} into the resolved hooks dir. Gitfile-aware (reads gitdir: out of .git when it is a file).
self-test.sh	End-to-end proof: writes 8 MiB of /dev/urandom as a fake video, archives at PART_SIZE=4m, asserts the manifest hash, asserts archive idempotency, moves the source away, extracts, compares hashes, asserts extract idempotency, then corrupts a part and asserts the extractor rejects it . Exits non-zero on any failure.
git-hooks/post-commit	exec scripts/archive-videos.sh \$ROOT
git-hooks/post-checkout	exec scripts/extract-videos.sh \$ROOT (also fires on clone)

self-test.sh is a genuine **mutation-paired** check in the §1.1 sense: the corruption case proves the verifier can FAIL. It is the only test in the submodule.

Manifest format (# video-archive-manifest v1), as found on disk:

```
# video-archive-manifest v1
video_sha256=345a743...d03f
video_size=1871981557
archive_sha256=67ee3b7...3bf714
part_size=50m
<sha256> <part filename>      ← one line per part
...
parts=36
```

1.5 docs/ and upstreams/

docs/superpowers/specs/2026-08-28-video-archive-pipeline-design.md is the design record for §1.4 — goal, layout, manifest format, decisions (tar+split, 50 MB parts “GitHub’s recommended ceiling”, SHA-256, git-

hook triggers), error handling, and an explicit **Out of scope** section: “PDFs and other chapter files are untouched” and “Nothing is committed or pushed by the scripts.”

upstreams/GitHub.sh is one line: export
UPSTREAMABLE_REPOSITORY="git@github.com:milos85vasic/
workshop_curriculum.git" — the §11.4.36 upstream-recipe convention.
There is exactly one upstream.

1.6 README.md — stated purpose

The entire file is:

```
# The Workshop Curriculum
```

Tbd.

There is no stated purpose. The submodule’s own commit history is six commits (286e011 Initial commit → ae4be7f “Revise README title and add placeholder text” → 66248e1/960b0f9 the archive pipeline → two Auto-commits). The only articulated intent for this submodule anywhere in the tree is the Gemini notes quoted in §1.3 and the operator’s brief that prompted this survey.

§2 · ai_interviewing/ — the reference implementation

This is the module the operator named as the structural model. It is a git submodule (git@github.com:milos85vasic/ai_interviewing.git, gitlink ed73d855, clean) with **915 tracked files**. Its docs/ tree is 206 MB.

Two things must be separated up front, because the operator’s brief conflates them:

- **ai_interviewing/ the corpus** — Markdown source + Mermaid diagrams + a four-format export pipeline. This is the *content* layer.
- **ai_interviewing/platform/ the app** — a Go backend + Angular SPA that ingests that corpus into SQLite and serves it as a browsable, drillable curriculum. This is the *delivery* layer.

The workshop needs both, and they are cleanly decoupled: the platform’s only input contract is “a directory of numbered Markdown files with diagrams/*.png beside them”.

2.1 Curriculum structure on disk

```
ai_interviewing/
├─ README.md / .html / .pdf / .docx      ← the corpus index,
four formats
├─ AGENTS.md  CLAUDE.md  GEMINI.md  QWEN.md ← the four governance
carriers
├─ assets/theme.css                      ← export theme
├─ scripts/                             ← the CONTENT
pipeline (§2.4)
├─ upstreams/GitHub.sh
├─ docs/
│   └─ README.md
│   └─ interview-preparation/
│       └─ 00-Overview-and-Taxonomy.md
│       └─ areas/NN-slug.md              ← 34 files
│           └─ (areas/)diagrams/*.mmd|png|svg
│   └─ company-research/NN-slug.md        ← 5 files
│   └─ project-deep-dives/NN-slug.md      ← 37 files
│   └─ improvement-plans/NN-slug.md       ← 37 files
│   └─ validation-plans/NN-slug.md       ← 37 files
│   └─ */diagrams/
└─ platform/                             ← the DELIVERY layer
(§2.2–§2.6)
```

Measured file counts under docs/:

Extension	Count	What it is
.md	156	source of truth (150 numbered + 6 README.md)
.mmd	159	Mermaid diagram sources
.png	159	rendered diagrams (1:1 with .mmd)
.svg	101	bonus vector renders (not 1:1 — see §2.4)
.html	67	export
.pdf	68	export (67 + the root README.pdf)
.docx	67	export

The naming/manifest convention that makes it browsable is a filename convention, not a manifest. There is no `index.json`, no front-matter, no TOC file. Everything is derived:

- `^(\d+)-(.)\.md$` — the leading number is the **ordinal**, the remainder is the **slug** (`ingest.go:reFile`). Files that do not match get ordinal 999.
- `README.md` in any content directory is **skipped** — treated as navigation, not a learning module (`ingest.go:collectDocs`).

- The first `^#` heading becomes the **module title**; the first paragraph becomes the **summary**; `wordCount/200 + 1` becomes **read minutes**.
- `^##` sections become **lessons**, in document order.
- `^#### (Q\d+)\. (.+)\$` items become **flashcards** (prompt = the Q line, answer = the body that follows).
- `] (diagrams/...)` links are rewritten to `/api/assets/<module-slug>__<file>` and the PNG is copied into the served assets dir.
- **Cross-file joins are by basename.** A module built from `project-deep-dives/17-helixllm.md` automatically acquires the *improvement plan* at `improvement-plans/17-helixllm.md` and the *validation plan* at `validation-plans/17-helixllm.md`. This is the entire mechanism for attaching parallel document kinds to a module — worth copying, because it is how a workshop chapter would attach a transcript, a code walkthrough, and an exercise set.
- Authored question banks join by **track + ordinal**: `platform/content/mcq/<track-slug>-<NN>.json` and `platform/content/short/<track-slug>-<NN>.json` — **34 files each**, one per area.

Tracks are **hardcoded in Go**, not discovered (`ingest.go, Run()`): three literals (`interview-preparation`, `employer-dossier`, `project-deep-dives`) each with a slug, title, subtitle, Material-icon name, and source directory. Domain grouping is also hardcoded, as a switch on the ordinal (`domainForArea`). **Adding a track today requires editing and rebuilding Go.** This is the single largest structural obstacle to “extensible by a repeatable prompt”.

`platform/README.md` states the published totals: 3 tracks, 77 modules, 435 lessons, ~1,700 flashcards, ~473 short-answer, ~302 MCQ, 37 + 37 plans, curriculum `ai-curriculum v1.0.0`. **UNVERIFIED by this survey** — the DB was not opened and the server was not started; these are the module’s own claims.

2.2 Container story — there is none

This is the most important negative finding in §2.

```
$ find . -iname 'Dockerfile*' -o -iname '*compose*.yml' -o -iname 'Containerfile*'
./_tools/helixtranslate-container/Containerfile
./_tools/helixtranslate-container/Containerfile.translator
./submodules/constitution/scripts/sonarqube/compose/docker-compose.sonarqube.yml
```

Zero of those are in `ai_interviewing/`. The reference implementation ships **no Dockerfile, no Containerfile, no compose file, and no container script**. It runs as native host processes.

What it ships instead is a **six-script bash control plane** in `platform/scripts/`, sourcing a shared `_common.sh`. The operator’s “runnable via containers with mandatory bash scripts” requirement gets the *bash scripts* half for free and the *containers* half not at all.

Script	Lines	Behaviour
<code>_common.sh</code>	46	Exports every path and address; mkdir -p bin logs run data; helpers json_field, server_http_url, server_pid. No jq dependency — a sed one-liner reads the state file.
<code>build.sh</code>	14	go build -o bin/ aicur ./cmd/ server; then, only if frontend/ package.json exists, npm ci \\ \\ npm install + npx ng build -- configuration production. Degrades to an API-only build otherwise.
<code>start.sh</code>	51	Idempotent (already running (pid N) + exit 0). Clears stale pid/ state files. Builds if bin/aicur is missing. nohup with 11 explicit flags. Then health-polls: fails immediately

Script	Lines	Behaviour
		if the process dies , otherwise waits up to HEALTH_TIMEOUT (120 s, because a cold start re-ingests the corpus in ~20 s), printing “still starting (ingesting corpus)” at t=10.
stop.sh	27	SIGTERM → poll 20 × 0.5 s → SIGKILL with an explanatory message. Cleans pid + state files.
status.sh	24	RUNNING (pid N) / STOPPED, the actually bound URLs, and a live / api/health fetch. Names a stale pidfile explicitly.
restart.sh	5	stop.sh; sleep 1; exec start.sh
ingest.sh	5	bin/aicur -ingest-only ... — rebuild the DB without serving.

Ports, addresses and volumes (defaults from _common.sh + main.go):

Thing	Default	Override
Plain HTTP	:8099	AICUR_HTTP, flag -http
HTTPS + HTTP/3	:8443	AICUR_HTTPS, flag -https
Port fallback	walk forward 20 ports if busy	AICUR_PORT_FALLBACK, -port-fallback
Health timeout	120 s	AICUR_HEALTH_TIMEOUT

Thing	Default	Override
SQLite DB	platform/data/ curriculum.db	-db
Served diagram assets	platform/data/assets/ (158 PNGs present)	-assets
TLS certs (self- signed, generated)	platform/data/certs/	-certs
Angular build	platform/frontend/dist/ ai-curriculum/browser	-web
Corpus input	../docs	-docs
Question banks	../content/{mcq,short}	-mcq, -short
Plan inputs	../.../docs/ {improvement,validation}- plans	-plans, -validation
PID / state / log	run/server.pid, run/ server.json, logs/ server.log	-state

Two design points worth carrying forward verbatim:

- **The server publishes the address it actually bound** to run/server.json, and the scripts health-check *that*, not the address they requested. This is what makes the port-fallback safe.
- Everything generated is git-ignored (platform/.gitignore: frontend/node_modules/, frontend/dist/, frontend/.angular/, qa/e2e/node_modules/, bin/, data/curriculum.db*, data/assets/, data/certs/, logs/, run/, qa/evidence/). **qa/evidence/ does not exist on disk** — evidence is produced by a run, never committed.

How a user actually starts it (from platform/README.md):

```
scripts/build.sh      # backend + Angular
scripts/start.sh      # ingest on first run, then serve; waits for health
# → http://localhost:8099 | https://localhost:8443 (HTTP/2 +
                        HTTP/3, self-signed)
```

2.3 UI/UX layer

Backend — Go 1.26.2, module github.com/milos85vasic/ai_interviewing/platform/backend. Direct dependencies (7): gin-gonic/gin v1.12, gin-contrib/cors, andybalholm/brotli, quic-go/quic-go v0.61 (HTTP/3), yuin/goldmark v1.8.5 (Markdown→HTML), modernc.org/sqlite v1.56 (**pure Go, no cgo**).

Package	Lines	Test lines	Role
cmd/server	156	—	flags, ingest-or-serve decision, state file, signal handling
internal/store	600	251	SQLite schema + all queries
internal/ingest	546	191	Markdown → curriculum
internal/server	367	104	dual HTTP/HTTPS listeners, TLS, HTTP/3, port fallback
internal/api	289	188	15 routes + SPA fallback

Frontend — **Angular 19.2** standalone components with signals and lazy routes. Dependencies are Angular + rxjs + zone.js only; no UI library, no state library, no HTTP client wrapper. Six feature components, all in `platform/frontend/src/app/features/`:

Route	Component	Lines
/	<code>home.component.ts</code>	123
<code>/module/:id</code>	<code>module.component.ts</code>	186
<code>/practice/:id</code>	<code>practice.component.ts</code>	248
<code>/plans</code>	<code>plans.component.ts</code>	146
<code>/search</code>	<code>search.component.ts</code>	61

Route	Component	Lines
/progress	progress.component.ts	64

Plus `core/api.ts` (88 lines — the typed HTTP client, session id in `localStorage`, `i18n` signal, theme toggle) and an 87-line `app.component.ts` shell. Styling is a single `styles.scss` built on the **OpenDesign --od-* token contract** with a `--lk-*` learning-kit layer on top, `light default + [data-theme="dark"]`.

Navigation is `/api/curricula/:slug/:version` returning the whole track → module tree in one response (no lesson or question bodies), rendered as cards; then per-module lazy fetches of `/lessons` and `/questions`.

Search — this is the weakest part of the reference implementation. `store.Search` lives in the **private** `ai_interviewing` repository at `platform/backend/internal/store/store.go` (function `Search`, around line 501). **Its query text is private content and is deliberately not reproduced here** — read it at that path if you need the exact statement.

Described rather than quoted: it is a single hand-written statement that UNION ALLs three content tables (modules, lessons, questions), matching each with a bound LIKE pattern against a small number of short columns and truncating the returned excerpt to a fixed prefix length, with a bound row limit.

That is a four-way LIKE '%q%' union, capped at 40, **with no ranking, no scoring, no FTS index, no stemming, no snippet-around-match, and no semantic component whatsoever**. Lesson *bodies* are not searched at all — only titles. The UI (`search.component.ts`) debounces 220 ms, requires ≥2 characters, and renders flat cards; **there is no autocomplete, no typeahead, no keyboard navigation, and no result grouping**. A repository-wide grep for `deepwiki|autocomplete|typeahead|fts5` found **no implementation anywhere in the tree**.

The full API surface (`api.go:Router()`), for reference when specifying the workshop's:

GET /api/health	GET /api/search?q=
GET /api/curricula	GET /api/i18n/:lang
GET /api/curricula/:slug/:version	GET /api/langs
GET /api/modules/:id	POST /api/progress
GET /api/modules/:id/lessons	GET /api/progress?
session=	
GET /api/modules/:id/questions[?kind=]	GET /api/assets/:name

```
GET /api/modules/:id/plan[?kind=]          (SPA fallback for
everything else)
GET /api/modules/:id/plan/download[?kind=]
GET /api/plans[?kind=]
```

SQLite schema (store.go:schema), 10 tables: curriculum (UNIQUE slug+version), track, module, lesson, question, asset, plan (UNIQUE module+kind), ui_string, content_i18n, progress (UNIQUE session+type+item). Four indexes. WAL, foreign_keys=ON, SetMaxOpenConns(1).

Two properties the workshop should inherit deliberately:

- **Curriculum versioning.** curriculum(slug, version) is unique, and the frontend pins {slug: 'ai-curriculum', version: '1.0.0'} in one constant. New content ships as new rows; the app selects by slug+version. No code change to swap or upgrade content.
- **i18n-readiness with zero translations.** ui_string(lang, key, value) and content_i18n(entity, entity_id, field, lang, value) exist and are wired to /api/i18n/:lang + /api/langs, with a language switcher in the UI. The platform is English-only today; adding a language is seeding rows.

2.4 Content pipeline — source material → published curriculum

Two pipelines, in series, with different owners.

Pipeline A — corpus build (ai_interviewing/scripts/, 5 scripts + 2 configs). build-all.sh is the deterministic, re-runnable entry point and does exactly three things:

1. render-diagrams.sh docs — every *.mmd → mmdc -w 1600 --scale 2 -b white → PNG, then a second pass → SVG. That script's header explains the choice; **the header text is private content and is not quoted here** (read it in the private ai_interviewing repository at scripts/render-diagrams.sh). The reasoning, restated independently: the §11.4.168 export check counts embedded raster images in the produced PDF, so a diagram embedded only as vector art satisfies the reader but scores zero on that check. Raster is therefore the load-bearing format and vector is supplementary. Failures are collected and reported, not swallowed.
2. export-doc.sh <file.md> for README.md and every docs/**/*.*md — pandoc -f gfm+smart -t html5 -s --css assets/theme.css --embed-resources --toc --toc-depth=2 → self-contained HTML; weasyprint that HTML → PDF; pandoc -t docx → DOCX. Title = first ^# heading.

3. `validate-exports.sh` . — \$11.4.168 independent validation (\$2.5).

Toolchain: `pandoc`, `weasyprint`, `mmdc`, plus `poppler` + `tesseract` for validation. **All five are present on this host** (\$3.6).

Pipeline B — ingest (`platform/internal/ingest`, 546 lines).

Deterministic and re-runnable: `store.Reset()` wipes content tables (progress is preserved) and rebuilds from source. `RunFull()` takes seven paths and produces the whole curriculum. Per module it renders Markdown with goldmark (GFM + `Typographer`, `WithUnsafe()`), splits into lessons and flashcards, rewrites and copies diagram assets, then joins the four optional sidecars (MCQ, short-answer, improvement plan, validation plan). Malformed authored items are **skipped with a `stderr` note, never inserted broken** — e.g. an MCQ with `correctIndex` out of range, or a short-answer with an empty prompt or answer.

There is no transcode step, no media handling, and no index-building step of any kind. Pipeline B’s total media awareness is “copy a PNG and rewrite its link”.

2.5 Test / validation approach and the evidence it produces

Three banks, aggregated by `platform/qa/helixqa/run-suite.sh` (40 lines) in the **private** `ai_interviewing` repository. Its header states an anti-bluff rule; **the wording is private content and is not quoted here** — read it at that path. Restated in different words: a bank is credited as passing only when it actually terminated successfully *and* left evidence behind, so neither a skipped bank nor an assertion-free run can score as a pass. Evidence root is `platform/qa/evidence/` (git-ignored, absent until a run).

Bank	Command	Content
1 · unit + integration	<code>cd backend && go test ./...</code>	734 lines of Go tests across <code>store / ingest / api / server</code>
2 · challenges	<code>qa/challenges/api-challenges.sh</code> (154 lines)	24 evidence-gated live assertions
3 · e2e + responsive	<code>cd qa/e2e && npx playwright test</code>	9 tests × 2 viewports = 18

Bank 3 **SKIPs with a reason** when `qa/e2e/node_modules/@playwright` is absent — never a faked pass. Playwright config uses the **system Chrome channel** (no chromium download), `baseURL` from `AICUR_URL` (default `http://localhost:8099`), projects `desktop (1280×900)` and `mobile (Pixel 5)`, HTML report to `../evidence/e2e/report`, `screenshot: only-on-failure`, `trace: retain-on-failure`. The config comment cites §11.4.190 for the dual-viewport requirement.

The 24 challenges (C1–C24) are the pattern most worth copying, because each one **asserts a real runtime property and prints the observed value**: `health ok=true`; curriculum title+version string equality; `tracks == 3`; `modules ≥ 60`; `questions ≥ 1600`; MCQ integrity via direct SQL (0 malformed across N modules); a live MCQ endpoint where every item has 4 choices and an in-range index; rendered lesson HTML with **no raw '####' leak**; a diagram asset served with an `image/` content-type; **Brotli** (content-encoding: br); **HTTP/3** advertised via `Alt-Svc: h3=`; `HTTPS+H2`; SPA `<app-root>` present; SPA deep-link fallback; a progress POST→GET round-trip; search returning `≥1` hit for reconciliation; `≥10` i18n strings; short-answer integrity; `plans ≥ 37` with slug+wordCount; plan download asserting content-type: `text/markdown`, content-disposition: `attachment`, and that the body starts with `"# "`; the same for the validation kind; and `≥9` modules in the newest domain.

The 9 e2e tests: home loads with real stats · open a module and read a rendered lesson · flashcard reveals its answer · short-answer reveals + self-grades · plans hub browses + exposes a download · plans hub toggles kind and downloads it · quiz scores a real MCQ · search returns grounded results · theme toggle switches light/dark.

A real portability defect to avoid inheriting. `api-challenges.sh:11` is:

```
SQLITE="${SQLITE_BIN:-<home>/Library/Android/sdk/platform-tools/
sqlite3}"
```

That default is a **macOS** path. On this Linux host `sqlite3` resolves to `<home>/Android/Sdk/platform-tools/sqlite3`, so **7 of the 24 challenges (C6, C7, C18, C19, C24 and the DB lookups feeding C8/C9) silently degrade to [FAIL] here unless SQLITE_BIN is exported**. `scripts/audit-hardcoded-paths.sh` exists at the umbrella root for exactly this class of bug; `ai_interviewing/` is a submodule and is not swept by it.

2.6 Existing search / indexing / LLM integration in ai_interviewing

Search: the SQL LIKE union of §2.3. Nothing else. **Indexing:** none. No FTS5, no vector table, no embeddings, no external index. **LLM integration: none.** There is no LLM call, no provider config, no API key handling, and no chat endpoint anywhere in ai_interviewing/. The HelixLLM, HelixAgent, LLMPProvider and LLMOrchestrator material in docs/project-deep-dives/ is **prose only** — interview-preparation writing *about* those systems, with no code here.

So of the operator's headline requirements, the reference implementation supplies **zero** of: semantic search, cross-modal search, cross-linking, autocomplete, ask-an-AI, LLM bridging, containers, and any media handling. It supplies **all** of: the content→SQLite→SPA architecture, the filename-convention curriculum model, the bash control plane, the versioning and i18n model, the four-format export pipeline, and the evidence-gated three-bank test discipline.

§3 · Reusable technology already in this tree

3.1 Lumen — semantic search

What it is here. Lumen is a **Claude Code plugin/MCP tool**, not a library or a service this repository owns. lumen on PATH is `<home>/local/bin/lumen`, a 2,505-byte bash launcher (“Managed by Claude Code”) that resolves `uname` → `lumen-linux-amd64` and `sort -V` picks the newest version under `~/.claude-shared/plugins/cache/*/lumen/*/bin/`. It currently resolves to **v0.0.41** at `~/.claude-shared/plugins/cache/claude-plugins-official/lumen/0.0.41/bin/lumen-linux-amd64`. `LUMEN_BIN` overrides. Go source for that version is on disk.

Nothing in this repository declares Lumen. There is no `.mcp.json` at the repo root (and `.mcp.json` is git-ignored). `.claude/settings.json` enables only `ashlr@ashlr-marketplace`; `.claude/settings.local.json` merely allow-lists two lumen MCP tool names. `.opencode/`, `.crush/`, `.kimi-code/` contain zero lumen references. The two live declarations are **outside** the repo:

- user-global `~/.claude.json` → `mcpServers.lumen = {"type":"stdio","command":"<home>/local/bin/lumen","args":["stdio"]}` → the `mcp__lumen__` tools;

- the plugin's own `plugin.json` →
`${CLAUDE_PLUGIN_ROOT}/scripts/run_studio`, enabled in `~/.claude-claude4/settings.json` → the `mcp__plugin_lumen_lumen__*` tools.

The plugin also installs a `PreToolUse` hook on `Grep|Bash` — the source of the “use `mcp__lumen__semantic_search` instead” nudges seen throughout this session.

Backend and model. Backend `ollama` at `http://localhost:11434 (/api/embed)`; `lmstudio` at `:1234` is the alternative. Model **`ordis/jina-embeddings-v2-base-code`** (768-dim). `~/.config/lumen/config.yaml` does **not** exist and every `LUMEN_*/OLLAMA_*` export in `~/.bashrc` is commented out, so everything falls through to built-in defaults.

Index store. `$XDG_DATA_HOME/lumen` → `<home>/local/share/lumen/`, which holds **764 index directories**. The directory name is `sha256(projectPath + "\0" + model + "\0" + IndexVersion)[:16]` with `IndexVersion="3"`. For this project that resolves — and was confirmed on disk — to `<home>/local/share/lumen/21bf1507a8925bcf/`, `index.db` = 335,679,488 bytes plus `WAL/SHM/lock`. Consequence documented in both project scripts: **changing the model name silently starts a second index instead of updating this one.**

CLI surface. `lumen [completion|hook|index|purge|search|studio|version]`. `lumen search <query> flags: -b/--backend, --cwd, -f/--force, --max-lines, --min-score (-1..1), -m/--model, -n/--n-results (default 8), -p/--path, --summary, --trace`. **There is no JSON output flag — plaintext only.**

MCP surface (schemas read from `cmd/stdio.go` at v0.0.41; all three `ReadOnlyHint: true`):

Tool	Inputs	Outputs
<code>semantic_search</code>	<code>query (req) · path · cwd</code> <code>· limit (8) · min_score</code> <code>· summary · max_lines</code>	<code>results[] {file_path, symbol, kind, start_line, end_line, score, content}, reindexed, indexed_files, filtered_hint, seed_warning, stale_warning. Auto-indexes if stale/empty.</code>
<code>index_status</code>	<code>path · cwd</code>	

Tool	Inputs	Outputs
		project_path, total_files, indexed_files, total_chunks, last_indexed_at, embedding_model, stale
health_check	none (struct{})	backend, host, model, reachable, message

.lumenignore (repo root) excludes exactly one path: `_tests/evidence/`. The file's own rationale: 4,526 tracked files there (1,353 json / 1,167 txt / 1,142 png / 515 log) against 95 real source files, costing ~14 of the ~15 hours a full index needed. Everything else — all source, all submodules, all docs — is indexed. Skip precedence in `internal/merkle/ignore.go`: `SkipDirs` → `SkipFiles` → `.gitignore` → `.lumenignore` → `.gitattributes` → extension, so the repo `.gitignore` also constrains the index.

The repository's two Lumen operations scripts are substantial and worth reading before writing any indexing task:

- `scripts/lumen-reindex.sh` (17,175 B) — `[project-path] [--force] [--allow-gpu]`. Exit 0 indexed · 1 gave up after `MAX_ROUNDS` (40) · 2 bad usage · **3 refused (Vulkan backend)** · 4 backend unsafe · 5 prerequisite missing. Resolves backend/host/model in Lumen's own precedence order (`env` → `config.yaml` → the **existing index's** `project_meta.embedding_model` → `/api/ps` then `/api/tags` when unambiguous → documented fallback), refuses to start when `journalctl -u ollama` reports `library=vulkan`, and runs a **32-text batch distinctness probe** (32 distinct texts must yield 32 distinct vectors) before and between rounds, unloading a wedged runner via `POST /api/embed {keep_alive:0}`.
- `scripts/lumen-index-doctor.sh` (22,276 B) — `[project-path] [--require-live-backend]`. Exit 0 healthy · 1 corruption · 2 could-not-inspect. Opens every `index.db` **read-only**, brute-force solves `IndexVersion` (0..63) from a known triple rather than hardcoding it, derives vector width from three independent sources and **refuses a verdict if they disagree**, masks by the `sqlite-vec` validity bitmap, then runs distinctness + NaN/zero/L2-norm checks. Uses private

exit codes 20–23 internally so a Python traceback can never be misread as “corruption found”.

`.lumen-reindex.log` (git-ignored, 883 B) records three forced rebuilds and one hard failure where ollama was not listening (dial tcp 127.0.0.1:11434: connect: connection refused), followed by a runner unload. All three entries report backend `library=cpu` and batch probe OK: 32 distinct texts -> 32 distinct vectors.

The single most consequential finding for this spec

Lumen cannot index or query non-text assets. At all. There is no configuration knob.

`internal/chunker/languages.go:35-52` is a **compile-time extension allowlist**, passed straight into the filesystem walk at `internal/index/index.go:150` (`merkle.MakeSkipWithExtra(projectDir, chunker.SupportedExtensions(), extraSkip)`), so unlisted extensions are never read; `index.go:290-293` additionally **purges** stored records whose extension is not in the set. The complete list is:

```
.go .ts .tsx .js .jsx .mjs .py .rs .rb .java .c .h  
.cpp .cc .cxx .hpp .php .cs .swift .dart .svelte  
.md .mdx .yaml .yml .json
```

No image, audio, video, PDF, HTML, CSS, SVG, or binary format, and no env or config override exists anywhere in the source.

The operator’s requirement — “*semantic/contextual search across transcripts, code, diagrams, audio and video*” — is therefore **reachable through Lumen only for the .md and source-code projections of that material**. Every media asset must be reduced to text (transcript .md, diagram description .md, frame-OCR .md) with stable timestamped anchors before Lumen can see it. That reduction step does not exist and would have to be built.

3.2 LLM-provider bridging — what is actually here

There is no LLM provider-bridging layer implemented in this tree.

Searching for HelixLLM, HelixAgent, LLMPProvider, LLMOrchestrator as *code* returns only .md and .json files. Specifically:

Prose only (documentation, no implementation here):

- ai_interviewing/docs/project-deep-dives/ — 17-helixllm.md, 18-llmprovider.md, 19-llmorchestrator.md, 04-helixagent.md, 02-helixqa.md, 16-helix-flow.md, plus their improvement-plans/ and validation-plans/ mirrors and rendered exports and diagrams. These are **high-quality, code-grounded write-ups of external repositories** (each states which files were read via shallow clone), which makes them an excellent *design reference* — and still not an implementation.
- _content/products/{HelixLLM,HelixAgent,LLMPProvider,LLMOrchestrator,...}.md × 15 locales — website product briefs.
- _analysis/top20/*.readme.txt — snapshots of **other repositories'** READMEs. Most OLLAMA_BASE_URL / OPENAI_API_KEY / ANTHROPIC_API_KEY strings in this tree live here and belong to repos that are not checked out.
- _analysis/github-helix-others.json (**56 repos**) and _analysis/github-vasic-digital.json (**185 repos**) are machine-readable inventories of the two organizations. The technology the operator refers to is catalogued here and **not vendored**:

Repo	Language	Description (verbatim from the inventory)
HelixDevelopment/HelixLLM	Go	“Helix LLM - Local running super model”
vasic-digital/LLMPProvider	Go	“Shared LLM provider interface, 40+ provider adapters, retry, circuit breaker, health monitoring”
HelixDevelopment/LLMOrchestrator	HTML	“Headless CLI agent

Repo	Language	Description (verbatim from the inventory)
		management for LLM orchestration”
HelixDevelopment/HelixAgent	Go	“LLMs Agent”
vasic-digital/LLMsVerifier	Go	“Benchmark and verify LLMs”
vasic-digital/LLMGateway	Go	(no description)
vasic-digital/ {Embeddings,VectorDB,RAG,Memory}	Go	“Generic reusable Go module: digital.vasic.*”
vasic-digital/Courses-Creator	Go	“Courses creator toolkit”
vasic-digital/containers	Go	(no description) — see §4.6

Courses-Creator deserves the spec author’s attention: ai_interviewing’s own deep-dive (docs/project-deep-dives/22-courses-creator.md) documents it, from a real clone, as “*markdown in, professional video course out*” — a Go core-processor with a multi-provider LLM manager (OpenAI / Anthropic / Ollama, each self-registering only when its key is present), a DB-persisted job queue with four workers, local Bark TTS with an OpenAI-TTS cascade, FFmpeg assembly, three clients on one /api/v1 surface, and Docker Compose + nginx + JWT + Prometheus/Grafana. The same document also states, anti-bluff, that several advertised features are stubs and that provider “routing” computes a score and then ignores it. **Whether any of this is reusable is UNVERIFIED** — the repository is not present in this tree and was not fetched.

Actually present as code or config:

1. Ollama is the Lumen embedding backend and nothing else.

Referenced by scripts/{ollama-tune,ollama-vulkan-

remediation,lumen-reindex,lumen-index-doctor,setup-agents-wizard,rollback-agents-wizard}.sh. No chat or completion call anywhere.

2. **The only literal provider registry in the tree** is a five-entry Python dict at `_tools/review_translation.py:63-68` — {groq, openrouter, mistral, zhipu, cohere} → (endpoint, key env var, default model). It **excludes** OpenAI, Anthropic, Gemini and Ollama. Mirrored at `submodules/constitution/scripts/translation/review_translation.py`.
3. **A translation pipeline that shells out to hosted APIs.** `_tools/translate-pipeline.sh:60-63` — primary mistral/mistral-large-latest, fallback groq/llama-3.3-70b-versatile, overridable via `TRANSLATE_PRIMARY_PROVIDER/_MODEL`. `_tools/gen/translate_home.py` hardcodes the Groq and Cerebras chat-completions URLs. The `translate_ui_*.py` family invokes the container with `-provider zhipu -model glm-4.5-flash`.
4. **_tools/gen/data/verified_models.db** (12,288 B) — a real SQLite file with table `verified_models(provider_id, name, verification_status, can_see_code, overall_score, capabilities, pricing, verified_at)` and **3 rows**: gemma-4-31b/cerebras, allam-2-7b/groq, codestral-2508/mistral, all verified, scores ~0.906–0.919, dated 2026-08-06. It is an **artefact produced by** the external unified-translator engine, not by code in this tree.
5. **Two real Go LLM clients — benchmark-only, inside the constitution submodule.** `submodules/constitution/submodules/helix_perf_cache/pkg/backend/llamacpp.go` (a working llama.cpp HTTP client, /health, completion with `cache_prompt`, /tokenize, default `http://localhost:18434`) and `anthropic_promptcache.go` (a working Anthropic Messages client honouring `ANTHROPIC_BASE_URL / ANTHROPIC_MODEL / ANTHROPIC_API_KEY | ANTHROPIC_AUTH_TOKEN`). Both are wired only into `cmd/perfbench/main.go`. **These are the closest thing in the tree to a local/external provider bridge, and they are a prompt-cache benchmark, not a routing layer.**
6. `submodules/constitution/scripts/llm-alias-health/` bridges to an external `claude-alias-probe` binary that is **not in this tree**; it cannot run as-is.

TLS / CA wiring for LLM egress: essentially absent. A repo-wide search for `SSL_CERT_FILE`, `CURL_CA_BUNDLE`, `REQUESTS_CA_BUNDLE`, `NODE_EXTRA_CA_CERTS`, `cacert`, `ca-bundle` returns **zero hits**. CA certificates for outbound LLM calls are handled *inside the container image* (`apk add ca-certificates && update-ca-certificates` in both Containerfiles). The only TLS code in the tree is `ai_interviewing/`

platform/backend/internal/server/server.go:94
(tls.Config{MinVersion: tls.VersionTLS12}, local server, unrelated)
and the constitution submodule's self-signed HTTP/3 helper.

Local model availability on this host (read-only inspection; nothing started, restarted or pulled):

- ollama serve is running (PID 3781) plus one runner (PID 889296).
- /etc/sysconfig/ollama contains one line:
GGML_VK_VISIBLE_DEVICES=-1 — the Vulkan remediation is **applied**;
ollama is **CPU-only**. /etc/systemd/system/ollama.service.d/ does
not exist, so ollama-tune.sh --apply has never been run.
- Model store is /var/lib/ollama/.ollama/models/ (not ~/.ollama/
models/, which has no manifests/).
- **Exactly two models are present locally, and both are
embedding-only:** ordis/jina-embeddings-v2-base-code:latest and
library/jina-embeddings-code-cpu:latest. They point at the **same
322,997,312-byte blob**; the -cpu tag differs only by num_gpu: 0.
- **No chat/instruct model is present** — no llama3, qwen, mistral,
gpt-oss. “Ask an AI a question and get an instant answer” **cannot
be served by a local model on this host today without pulling
one**, and pulling one was out of scope for this read-only survey.

Host-level provider bridging outside the repository (context, not a repository capability): ~/.claude-code-router/config.json declares roughly **40** OpenAI-compatible providers with Router.default = deepseek, deepseek-v4-pro, several routed through a local shim at http://127.0.0.1:3457/v1/chat/completions; and ~90 per-provider CLAUDE_CONFIG_DIR homes exist under ~/.claude-prov-helixagent, ~/.claude-prov-helixagent-native and ~/.claude-prov-helixllm-gateway. Their settings.json files have **empty env blocks and no mcpServers** — **UNVERIFIED** how those endpoints are actually supplied. ~/.config/helixcode/config.json exists beside a cookie_secret and was deliberately **not read**.

3.3 Design system — design-system/

The platform/README.md claim that design-system/learning-kit/ exists is **VERIFIED TRUE**.

design-system/	
├─ README.md	← does NOT mention
learning-kit/, motion/, or diagrams/	
├─ components-extended.css	24,425 B – brand-agnostic,
278 --od-* refs, no :root	
├─ brand-milosvasic/	milosvasic.css 24,384 B

```

(348 tokens) + DESIGN.md
├─ brand-vasic-digital/          vasic-digital.css 28,350 B
(285 tokens) + DESIGN.md + fonts/
├─ learning-kit/                ← the reusable curriculum
UI kit
|   └─ README.md                15,006 B – "OpenDesign Learning Kit",
lk- namespace over --od-*
|   └─ kit-tokens.css           6,565 B – 96 token lines
|   └─ learning-kit.css         31,715 B – cards, flashcards, quiz
options, progress rings, pills
|   └─ demo.html                17,348 B
├─ motion/                      animations.css, overlays.css,
motion.js, vendor/lottie-mini.js, 3 lottie JSONs
├─ fonts/                      fonts.css + Inter / Space Grotesk /
JetBrains Mono woff2 + OFL
├─ icons/icons.svg
├─ diagrams/                    32 product SVGs + _prompts/ (32
prompts) + build-and-generate.sh
└─ preview/                     3 HTML + 4 PNG proofs

```

This is the single most directly reusable asset for a workshop UI: it is framework-free CSS, already consumed by the Angular reference implementation (platform/frontend/src/styles.scss:40,91 layers --lk-* on --od-*), and already referenced by a governance gate (scripts/verify-all-constitution-rules.sh:172-173, OD_THEME_GLOBS / OD_TOKEN_GLOBS).

Two hazards: (a) **three on-disk copies of the kit exist** — design-toolkit/proposed/learning-kit/ and submodules/constitution/submodules/design-toolkit/proposed/learning-kit/ are byte-identical to the *.bak.20260827-183650 snapshots in design-system/learning-kit/, i.e. **one revision behind the live files**; (b) design-system/README.md's "Structure" block omits learning-kit/, motion/ and diagrams/.

design-toolkit/ (submodule, efd2c3fb = v0.2.2-6-gefd2c3f, clean) carries all four governance carriers plus agents/ (11 design-specialist recipes), docs/, evidence/ (with tokens/*.tokens.json for helix / milosvasic / vasic-digital), generators/ (dtcg-to-od.mjs, gen-marks.mjs, gen-tokens.mjs, one test), knowledge/, qa/ (check-tokens.mjs, run-checks.mjs, platform-metrics.mjs, lib/{apca,cam16,deltae,dnadist,platform}.mjs), and proposed/. Note: **mcp/ contains only INSTALL.md** — there is no MCP server implementation — and **submodules/ contains only PLAN.md**.

3.4 The Go generator — `_tools/gen/`

Module `vasic.digital/tools/gen`, **go 1.26**, **zero external dependencies** (no `require`, no `go.sum`), single package `main`, 15 `.go` files, `stdlib` only.

- **Entry point:**

```
gen -site <vasic.digital|milosvasic.ru> [-lang en] [-what all|
products|portfolio|home] [-root <repo>] [-out <dir>].
Deterministic footer year via -ldflags "-X main.buildYear=YYYY".
```

- **Two hardcoded sites:** `vasic.digital` (brand “Vasic Digital”, `Jekyll:false`, JSON-LD Organization) and `milosvasic.ru` (brand “Miloš Vasić”, `Jekyll:true` — pages emitted as Jekyll docs inheriting `_layouts/default.html`, JSON-LD Person).
- **There are no template files.** No `text/template`, `html/template`, `ParseFiles`, `ParseGlob`, or `.tmpl` anywhere. HTML is assembled with `strings.Builder/fmt.Sprintf` in `home.go` (25 KB), `product.go` (17 KB), `portfolio.go` (13 KB), `seo.go` (13 KB), `shells.go` (10 KB). The only `//go:embed` is `ui-i18n.json` in `i18n.go:17` (86,674 B). `markdown.go` is a hand-rolled Markdown renderer.
- **Inputs:** `_content/products/*.md`, `_content/portfolio/portfolio.json`, `_content/sites/*.home.json` (+ per-language), and `design-system/diagrams|icons`.
- **Localization:** `seo.go:41` `availableLangs()` discovers languages by scanning the repo root for `_content_<code>` directories → 14 non-EN + EN = **15 languages** (ar be de es fa fr hi ja kk ko ru sr tr zh). EN is canonical and unprefixed; non-EN routes to `products/<lang>/<slug>.html`, `portfolio/<lang>/index.html`, `<lang>/index.html`. RTL handled by `htmlDir` in `i18n.go`.
- **What it can already render:** product pages, a portfolio index, a home page, and — on `-what all` — `sitemap.xml` + `robots.txt` built by walking the emitted tree. A source comment cites “~288 translated product/portfolio pages”; that figure is **UNVERIFIED** (not an executed measurement).
- **Tests: yes** — 6 `_test.go` files, **30 Test* functions** (`data_test 5`, `home_test 2`, `markdown_test 7`, `portfolio_test 4`, `product_test 4`, `seo_test 8`). Not executed in this survey.
- `_tools/gen/build.sh` pins `buildYear` (`SOURCE_DATE_EPOCH` → last commit year → now), **copies** `design-system` assets into `<site>/assets/od/`, runs `gen` for both sites, then rebuilds `milosvasic.ru/_site` with `jekyll build`.

Assessment for the spec: this generator is tightly bound to the two portfolio sites (hardcoded site table, no template layer, portfolio/product-shaped data model). It is a **precedent for “Go + stdlib +**

deterministic + tested”, not a component a curriculum renderer can call. The reusable *idea* is availableLangs()-style discovery-by-directory and the i18n JSON split.

3.5 Container tooling conventions — podman vs docker

Measured on this host: podman = /usr/bin/podman, **version 5.7.1**, rootless=true, storage driver overlay. **docker is not installed** (command -v docker → not found).

The repository’s convention is selection by *remote hostname*, never by probing the local host, and it is **triplicated with no shared helper** (there is no lib/container-runtime.sh anywhere):

_tools/helixtranslate-container.sh:43-46 (canonical):

```
HOST="${HT_HOST:-thinker.local}"
if [ -n "${HT_RUNTIME:-}" ]; then RUNTIME="${HT_RUNTIME}"
elif [ "$HOST" = "amber.local" ]; then RUNTIME="docker"
else RUNTIME="podman"; fi
```

_tools/translate-fleet.sh:56 — local rt="podman"; ["\$host" = "amber.local"] && rt="docker" _tools/distribute-helixtranslate.sh:74 — rt=podman; ["\$H" = amber.local] && rt=docker (plus hardcoded podman build, podman save ... | ssh ... 'docker load', docker tag).

Hardcoded, no selection: _tools/helixtranslate-local.sh:32 is podman-only. _tools/helixtranslate-container/run.sh:11 takes the runtime as \$1.

The **only** command -v-based selection is scripts/ollama-tune.sh:294-309, which tries **docker first, then podman** — the opposite of the constitutional preference (§4.6).

The only container definitions in the whole tree are _tools/helixtranslate-container/Containerfile (runtime-only: alpine:3.20 + ca-certificates + a prebuilt static binary) and Containerfile.translator (the one actually built: multi-stage **cgo** build on golang:1.26-alpine with sqlite-dev, runtime alpine:3.20 + sqlite-libs, mkdir -p /data; cgo is required because the binary opens a go-sqlite3 verified-models store at startup). Its header records the distribution model: “*Built natively on thinker.local (linux/amd64), then replicated to amber.local via podman save | ssh ... docker load.*” **The HelixTranslate engine source is not in this tree** — _tools/

helixtranslate*.sh are invocation shims around the image
helixtranslate:cli. Whether that image exists locally is **UNVERIFIED**
(podman images was not run).

nerdctl, buildah, and the env var CONTAINER_RUNTIME have **zero occurrences** anywhere.

3.6 Host toolchain (measured with command -v)

Tool	Path	Relevance
podman	/usr/bin/podman (5.7.1, rootless)	containers
docker	MISSING	—
ffmpeg / ffprobe	<home>/bin/ (7.0.2- static)	media transcode / probe
pandoc	<home>/.local/bin/ pandoc	HTML + DOCX export
weasyprint	<home>/.local/bin/ weasyprint	PDF export
mmdc	<home>/.npm-global/ bin/mmdc	Mermaid → PNG/SVG
pdftotext	/usr/bin/pdftotext	§11.4.168 textual layer
tesseract	<home>/.local/bin/ tesseract	§11.4.168 OCR layer
go / node / npm / ruby / bundler / python3	system paths	build toolchains
sqlite3	<home>/Android/Sdk/ platform-tools/ sqlite3	note the unusual path (§2.5)
lumen	<home>/.local/bin/ lumen (launcher → v0.0.41)	semantic search
ollama	/usr/bin/ollama (running, CPU-only, embeddings only)	embeddings
ASR (speech-to-text)	MISSING	see below
yt-dlp	MISSING	—

`/usr/bin/whisper` is **NOT OpenAI Whisper**. It is whisper-1.3.1-alt1.noarch, a GPL-3 Python utility by Lorenzo Paderi (`/usr/share/whisper/whisper/window.py`). `python3 -c "import whisper"` → `ModuleNotFoundError: No module named 'whisper'`. **There is no ASR tooling on this host**. Transcribing chapter 1 requires installing one (openai-whisper, faster-whisper, whisper.cpp, or a hosted API) — and note that ollama here is **CPU-only** with **no GPU path**, so local ASR of a 115-minute recording is a multi-hour CPU job that must respect the §12.6 60 % RAM cap.

3.7 The umbrella test/evidence conventions (`_tests/`)

The workshop’s “real test coverage with machine evidence” requirement has an existing house pattern, documented in `_tests/GATES.md` and `_tests/TEST-TYPES.md`:

- **Mutation-paired self-validation as a golden-good / golden-bad pair.** `bash _tests/export/self-validate.sh` drives `_tests/export/validate-pdf.js` over `golden-good.pdf` (expect PASS) and `golden-bad.pdf` (expect FAIL) and **exits 0 only if good=PASS and bad=FAIL**. `bash _tests/visual/self-validate.sh` does the same for the visual oracle. This is §1.1 made runnable, and it is the template any new gate must follow.
- **Gate literals are greppable.** Each satisfying script echoes its CM-* literal (CM-HOST-RENDERED-UI-VISUAL-PROOF §11.4.170, CM-EXPORTED-DOC-VISUALLY-VALIDATED §11.4.168, CM-OPENDESIGN-UI-SYSTEM §11.4.162), so a reviewer can go from a governance document to the runnable check.
- **`_tests/GATES.md` openly declares CM-OPENDESIGN-UI-SYSTEM NOT green**, lists its residual coverage gaps, and instructs the reader not to claim them. That honesty posture is the expected form for the workshop’s own gate documentation.
- Evidence lands under `_tests/evidence/...` (and is the one path `.lumenignore` excludes). `ai_interviewing` uses `platform/qa/evidence/`. `scripts/test-setup-agents-wizard.sh` uses `.test-evidence/<UTC-timestamp>/` with `results.tsv` + `run.log` + `summary.json` — **73 such directories exist at the repo root**, the newest written during this session.

3.8 SpecKit state

`.specify/` is initialized: `templates/` (spec, plan, tasks, checklist, constitution), `scripts/bash/` (create-new-feature.sh, setup-plan.sh, setup-tasks.sh, check-prerequisites.sh, common.sh), `workflows/`

speckit/workflow.yml, integrations/ manifests for claude / kimi /
opencode / speckit, and the superspec extension (also a submodule at
submodules/superspec, third-party WangX0111/superspec, **outside the
owned-submodule set**).

.specify/memory/constitution.md (v1.0.0, updated 2026-08-31) is the
SpecKit project constitution and already encodes five core principles
— Evidence-Based Claims, Governance Fidelity, Isolation by Default,
Comprehensive Documentation, Quality Over Speed — plus the project
structure, the CI/CD policy, the testing commands and the deploy story.
A specify run will inherit these. .specify/feature.json is currently
untracked.

§4 · Binding constraints the spec must respect

All quotes below are verbatim from <ext-volume-host>/Projects/
vasic/submodules/constitution/Constitution.md (11,689 lines). Line
numbers are that file's.

4.1 §11.4.156 — the CI prohibition · release blocker

Heading, L9552: ### §11.4.156 – All CI/CD automation (GitHub
Actions / GitLab pipelines / equivalents) **MUST** be disabled (User
mandate, 2026-06-15)

L9558 — “Every repository this Constitution governs — the main
repo, this constitution submodule, and every owned + nested
submodule we author and push — **MUST** ship with ALL server-
side CI/CD automation **DISABLED**. No push to any owned
upstream may trigger a GitHub Actions run, a GitLab pipeline, or
any equivalent provider-side automation (Jenkins, CircleCI,
Travis, Drone, Woodpecker, Bitbucket Pipelines, Azure Pipelines,
or any on: push / schedule / workflow_dispatch workflow).”

L9560 — “**(A) Zero active CI at the repository root.** No active
.github/workflows/*.yml|*.yaml, no .gitlab-ci.yml, no .gitlab/
** pipeline include, nor any equivalent provider config may exist
at the ROOT of any governed repository/submodule — the only
location a provider executes. **(B) Disabled means a push
triggers ZERO runs.**”

L9566 — “Non-compliance is a release blocker regardless of context. No escape hatch — no `--allow-ci`, `--enable-workflow`, `--keep-pipeline`, `--remote-ci-OK`, `--ci-exempt` flag.”

Applied to this work. `workshop/` is an owned submodule and is therefore governed. **No workflow file of any kind may be added to `workshop/`, and none may be added to the umbrella root.** Measured current state: root holds only `.github/workflows/ci.yml.disabled`; `milosvasic.ru/.github/workflows/pages.yml` is **ACTIVE** and is a documented deviation that must not be touched (§4.7); `vasic.digital` has no `.github/workflows/` directory at all. `workshop/` has no `.github/` directory.

Do **not** propose an Override §11.4.156. The root `Constitution.md` records at L114-129 that one was sought on 2026-08-27 and is structurally unavailable, drawing the distinction the spec must preserve:

“An override claims a clause no longer applies; these admit the clause applies and is knowingly unmet, with the reason on the record.”

4.2 §11.4.157 — carrier lockstep

Heading, L9568. **L9576:**

“`GEMINI.md` is a FIRST-CLASS governance context carrier, EQUAL to `CLAUDE.md` / `AGENTS.md` / `QWEN.md` — NEVER an optional or best-effort sibling. Every governance addition or edit MUST land in ALL FIVE carriers in lockstep: the canonical `Constitution.md` PLUS each per-agent mirror `CLAUDE.md` + `AGENTS.md` + `QWEN.md` + `GEMINI.md` ... in the SAME change-window, each with its synchronized `.html/.pdf/.docx` exports per §11.4.65.”

Clause (D), same line: “*The lockstep binds the consuming project’s repository-root context files ... a project that maintains the other three but not `GEMINI.md` is non-compliant.*” **L9582:** “No escape hatch — no `--skip-gemini-md`, `--gemini-optional`, `--gemini-lag-OK`, `--four-carrier-suffices` flag.”

Note for the spec author: the anchor says **five** (the canonical corpus plus four mirrors). The repository’s own carriers call it “four-carrier lockstep”, counting only the mirrors. Both are the same obligation; use the anchor’s wording.

Applied: workshop/ currently has **none** of the four carriers, and CONTINUATION.md records this as gap **G7 at 20/24**. Another agent is closing it. Any governance text the workshop spec introduces must land in all four workshop/ carriers *and* any umbrella-root carrier it touches, in one change-window.

4.3 §12.10 — CONTINUATION.md sync

Heading, L10079: ### §12.10 Continuation document – sacred invariant. **L10092-10096:**

“A single, canonical, machine-readable handoff document — docs/CONTINUATION.md — must always reflect the live state of the project. Any agent ... must be able to resume work **exactly where the previous session left off** by reading this single file.”

L10101-10105:

“1. docs/CONTINUATION.md **MUST exist** at the project root. Its absence is a release blocker. 2. **Every non-trivial state change** MUST update this document in the same commit as the work itself.”

Applied: this repository resolves the anchor’s internal path inconsistency in favour of the repository root — CONTINUATION.md — and says so explicitly in its own header. Every non-trivial workshop commit must update it in the same commit, including the Last-Updated and Synced-Commit fields, which are machine-read by scripts/continuation-check.sh (a 30,425-byte checker with a --prove-failure mode and exit codes 0/1/2 where UNDET forces 2).

4.4 §1.1 — mutation-paired gates · “*the single most important rule*”

Heading, L147: ### 1.1 False-positive immunity is an invariant. **L149-152:**

“A test that always returns PASS because its regex never matches, its input path is wrong, its assertion target does not exist, or its comparison is tautological is **worse than no test**. Every new gate MUST be paired with a mutation entry in the project’s meta-test harness that:”

L160-161: *“If the mutation round does not turn PASS → FAIL, the gate is a sham and must be rewritten.”* **L163-164:** *“This is the **single most important rule in this Constitution**. Every subsequent §11.4.x clause is downstream of it.”* **L145:** *“A change without all four layers of coverage is not ready to merge.”*

Applied: every gate the workshop introduces needs a paired mutation. The existing templates to copy are `_tests/export/self-validate.sh` and `_tests/visual/self-validate.sh` (golden-good must PASS **and** golden-bad must FAIL), plus `workshop/scripts/self-test.sh`, which already does this for the archive pipeline by corrupting a part.

4.5 §11.4.6 — anti-bluff / no guessing

Heading, L632. **L634-639:**

“Tests, gates, status reports, closure narratives, commit messages, and any operator-facing text **MUST NOT** use words like likely, probably, maybe, might, possibly, presumably, seems, appears to, guess, seemingly, apparently, perhaps, supposedly, conjectured, or their synonyms when describing CAUSES of test failures, system behaviour, or fix effectiveness.”

L649-651: *“Every ‘X likely caused Y’ sentence in the codebase or documentation is a §11.4.6 violation.”* The two permitted outs (L641-647): **prove the cause** with captured forensic evidence and state it as fact, **or** explicitly mark **UNCONFIRMED + PENDING_FORENSICS** with a tracked-task ID.

4.6 Containers — §11.4.76, §11.4.161, §12.3 · new to this work

The operator’s “runnable via containers” requirement pulls in three anchors that no existing module in this repository satisfies.

§11.4.76 (L7113) — Containers-submodule mandate. L7121:

“For ANY containerized workload — Docker, Podman, Qemu, Kubernetes, container-backed emulators ... — every consuming project **MUST**:”

...(1) use `https://github.com/vasic-digital/containers` as the authoritative library, (2) install it **as a git submodule**, (3) **L7125** — *“Tests, CLI doctor commands, and local-development workflows that depend on Postgres / Redis / OTel / message-brokers / emulators **MUST***

invoke the Containers Submodule's pkg/boot + pkg/compose + pkg/health APIs to bring infra up automatically. Operators MUST NOT be required to manually start podman machine/docker compose up before tests (the **on-demand-infra invariant**), and (4) extend, never reimplement.

§11.4.161 (L9681) — Rootless container runtime mandate. L9685:

“Every project governed by this Constitution MUST use **Podman in rootless mode** (or an equivalent rootless container runtime) for ALL containerized workloads. Docker in rootful mode, sudo, or any escalation to root for container management is **STRICTLY FORBIDDEN** unless the target platform genuinely has no rootless container option AND that platform constraint is documented per §11.4.112.”

L9687: *“the vasic-digital/containers Submodule MUST be used as the **sole container orchestration layer** — every container operation MUST go through the Submodule's pkg/boot / pkg/compose / pkg/health primitives. No ad-hoc docker/podman commands outside the Submodule.”* **L9689:** a test that silently depends on an already-running daemon *“is a §11.4 PASS-bluff at the test-infrastructure layer.”*

§12.3 (L10038) — Container hygiene. L10040: *“Containers (Docker / Podman) the project owns or relies on MUST:”* (1) carry an explicit memory limit, (2) set OOMPolicy=stop, (3) use exponential-backoff restarts, (4) rebuild clean-slate after a host crash.

Measured compliance state, stated plainly:

- vasic-digital/containers **exists as a repository** (Go, public — confirmed in `_analysis/github-vasic-digital.json`) and is **not a submodule of this repository** (`grep -i containers .gitmodules` → no match). Satisfying §11.4.76(2) is therefore a **new submodule addition**, an operator-authorized act.
- Every existing container call site in this tree is an **ad-hoc raw invocation**, and the `amber.local` branch selects **Docker unconditionally** — on its face contrary to §11.4.161. **UNVERIFIED** whether a documented §11.4.112 platform exception exists for `amber.local`; none was located.
- The host is podman-5.7.1-rootless with no docker, so the rootless mandate is satisfiable here today.

4.7 Test evidence, coverage, and script discipline

Anchor	Line	The binding sentence
§11.4.262 — machine-created evidence	L11424	<i>“every claim by the System that something WORKS / PASSES / IS VERIFIED / IS PRODUCTION-READY MUST be backed by MACHINE-CREATED, MACHINE-VERIFIABLE evidence — a file / verdict / measurement produced by the gate itself, cited by path + content-hash + timestamp, with a schema an independent verifier can parse.”</i> Forbids "I checked it works" / "looks fine" / "no error was reported". L11426 defines the three-gate coverage (CONFIRMATION / VALIDATION / VERIFICATION).
§11.4.169 — test-type coverage	L9951	<i>“Every project under this Constitution MUST be covered by the following CLOSED, ENUMERATED set</i>

Anchor	Line	The binding sentence
		<i>of test types ... with every PASS backed by rock-solid captured PHYSICAL evidence ... A green result with no falsifiable captured evidence is a §11.4 PASS-bluff regardless of which layer reports it.” The closed set: unit, integration, e2e, full automation, challenges, HelixQA, DDoS, security, stress+chaos, concurrency/ atomicity, race/ deadlock, memory, benchmarking.</i> <i>Operator mandate, verbatim: “Any work we do MUST START by writing the test! ... Bash scripts as well!</i>
§11.4.224 — TDD + coverage floor	L10736	<i>Coverage with TDD MUST BE close to 100%, not less than 85% !!!” — the only anchor stating a numeric floor, and it binds new bash scripts explicitly.</i>
§11.4.2 — recorded evidence	L519	<i>“A test that emits PASS without captured visual or</i>

Anchor	Line	The binding sentence
		<i>audio evidence of the user-visible feature actually working on the screen the user would see is a §11.4 PASS-bluff.</i>
		<i>“Every Bash / shell / POSIX-sh script ANYWHERE in a project ... MUST carry:”</i> an in-source documentation block (Purpose one-liner; Usage with all flags, env vars, positional args, examples) plus complete user guides/manuals.
§11.4.18 — script documentation	L1103	<i>“No documentation ever can be out of sync with its codebase!”</i>
§11.4.67 — shell parseability	L6188	A script MUST parse under the shell it is actually invoked with.
§11.4.234 — hook-validation script	L10989	Hook checks run as an explicit stage of a dedicated commit/push script, and the commit/push mechanism is ALWAYS unblocked.

Export and validation:

Anchor	Line	Obligation
		<i>“any other .md file that documents the project — guides, research notes, plans ... README files inside owned submodules — MUST also have .html + .pdf siblings, all three artefacts in sync at all times.”</i> Operator mandate L5962:
§11.4.65	L5978	<i>“Every time when Markdown document (file) is modified, its proper HTML and PDF versions MUST BE regenerated.”</i> §11.4.65 is HTML+PDF only — the string DOCX does not appear anywhere in its span.
§11.4.153(6)	L9516	<i>“the feature Status set MUST export to HTML + PDF + DOCX (this doc class ADDS DOCX to the §11.4.65 universal HTML+PDF export set; the other doc classes are unchanged).”</i>
§11.4.73	L6900	NOT an export anchor — it is <i>“Main-specification document versioning +</i>

Anchor	Line	Obligation
		<i>revision discipline”.</i> Do not cite it for the export mandate. (The corpus’s own cross-reference from §11.4.168 mislabels it.) <i>“Every generated/ exported document ... MUST pass INDEPENDENT validation by a review agent structurally separate from the generator ... NEVER the author/ generator self- checking its own output ... run on BOTH the source Markdown AND every exported artifact derived</i>
§11.4.168	L9927	<i>from it, ALWAYS — every export, not a sampled subset — across THREE layers, ALL of which MUST hold.”</i> Layers: CONTENT (L9929), TEXTUAL (L9931), FULL VISUAL (L9933, via pdftotext / pdfimages / pdftoppm→OCR), plus an anti-bluff validator (L9935). L9943: <i>“No escape hatch — no --skip-</i>

Anchor	Line	Obligation
		<i>visual-validation, --raw-source-in-pdf-ok, --textual-check-suffices, --self-validate-export, --diagram-source-as-text-ok flag exists.</i>

Other operational anchors:

- **§12.6 (L10049) — Memory-Budget Ceiling.** L10057: “*Project procedures MUST NOT use more than 60% of total system RAM.*” L10073: “*Heavy work MUST be wrapped in a bounded execution scope so the kernel OOM-kills only the scope.*” L10075: “**No escape hatch.** §12.6 has NO operator-facing override flag.” — **directly binding on ASR, transcode, and embedding jobs over a 1.87 GB video.**
- **§9.3 (L421) — Hardlinked backup is the standard.** L423: “*Any destructive operation without a fresh hardlinked backup is a Constitution violation regardless of how small the repo is or how confident the operator.*” Mechanics in §9.1 (L361-365): `cp -al .git <backup>/<repo>.git.mirror` **before touching anything**; §9.1.5 is the post-op gate. (Note the pre-existing `workshop-git-backup-2026-08-29/` directory at the repo root — evidence this practice is already applied to `workshop/`.)
- **§2 (L168) — “All commit and push work uses the project’s official multi-remote commit wrapper ... Direct `git commit`/`git push`/`git add` on the main repo is prohibited in normal workflow.”** **§2.1 (L187)** — every project on multiple providers MUST push to ALL configured upstreams. **§11.4.36 (L2971)** — when an `upstreams/` directory with `*.sh` recipes is present, `install_upstreams` MUST be invoked; L2989: skipping it “*is a §11.4.36 violation — the next push from that working tree will land on only one upstream.*” (`workshop/upstreams/GitHub.sh` exists; §11.4.29 at L2227 is the *snake_case naming* anchor, **not** the `upstreams` convention — a common miscitation.)
- **§11.4.32 (L2038) — post-Constitution-pull sweep.** “*Whenever a consuming project’s constitution submodule is fetched + pulled with a **ny** content change ... the consuming project MUST execute a full-project + recursive-submodule validation sweep BEFORE the new constitution HEAD is treated as canonical for any other work.*”

Implemented as `scripts/verify-all-constitution-rules.sh` → step 1 `scripts/verify-governance-cascade.sh`.

4.8 The local pre-push gates — every one must stay green

`scripts/pre-push-gates.sh` (21,743 B) is the **only enforcement that exists**; there is no server-side CI. `.git/hooks/` is untracked, so a fresh clone runs nothing until `bash scripts/pre-push-gates.sh --install`, and `git push --no-verify` bypasses it with no record.

`GATE_IDS=(E 0 1 2 3 4 5 6)`, in order:

id	name	exact command
E	§11.4.156(E) no active root CI config	<code>git ls-files \ grep -E '^\.github/workflows/.*\.ya?ml\$ \ ^\.gitlab-ci\.yaml\$' — must be EMPTY</code>
0	hardcoded path audit	<code>./scripts/audit-hardcoded-paths.sh</code>
1	Go unit tests (<code>_tools/gen</code>)	<code>cd _tools/gen && go test ./...</code>
2	hardcoding audit (builds the Go generator)	<code>bash _tools/audit-hardcoding.sh</code>
3	HelixTranslate reproducibility self-test	<code>bash _tools/translate/reproducibility-selftest.sh</code>
4	portfolio §1.1 data-integrity self-validation	<code>bash _tools/portfolio/self-validate.sh</code>
5	harness self-validation (visual §11.4.170 + export §11.4.168)	<code>bash _tests/run-harness-selfvalidation.sh</code>
6	Playwright (chromium), excluding the all-language crawl	<code>cd _tests && npx playwright test --project=chromium --</code>

id	name	exact command
		grep-invert 'all-language'

Gate E reads the **captured text**, not `$?` (under pipefail a no-match grep exits 1, and no-match is the PASS case). Untracked matching files raise an advisory WARNING only, because the commit wrapper runs `git add ..` Gates 5 and 6 **SKIP** when `_tests/node_modules/@playwright/test` is absent; gate 6 additionally needs `milosvasic.ru/_site/index.html` and `vasic.digital/index.html`. `PREPUSH_STRICT=1` turns an unmet precondition into FAIL. `--install hardlink-backs-up` any pre-existing hook and writes a thin shim that execs the tracked script, so edits take effect without reinstalling. **There is deliberately no flag that skips gate E.**

Env knobs: `PREPUSH_VERBOSE`, `PREPUSH_SKIP_SLOW`, `PREPUSH_STRICT`, `PREPUSH_ONLY`.

Implication: adding a curriculum platform must not break gates 1–6, and **must not add any file matching gate E’s probe** in the umbrella root. A workshop/ workflow file would not be caught by gate E (which only sweeps the umbrella’s own `git ls-files`) but would still violate §11.4.156, which governs “*every owned + nested submodule*”.

4.9 Two production websites — do not disturb

Measured:

- `milosvasic.ru/.github/workflows/pages.yml` — **present and ACTIVE** (1,333 bytes). CLAUDE.md records the material fact: `gh api repos/milos85vasic/milosvasic.ru/pages` returns `build_type: "workflow"`, so this workflow is the **sole publish path** for the live site — there is no `gh-pages` branch and no `docs/` folder, and the repository root is Jekyll *source*, so it cannot be served raw. `_tools/deploy-langs.sh` generates, commits and pushes source and then sleeps waiting for a rebuild; **it does not publish**. Operator directive on record: “*Make sure all pages websites work flawlessly! No website can be broken! All websites we have here are running deployed in production!*” **Do not disable, rename, or “fix” pages.yml.**
- `vasic.digital` — **no .github/workflows/ directory at all**; committed static HTML. Its Pages source is `build_type: "legacy"`, so every push still triggers a provider-side pages build and deployment run. Non-compliant at the **provider** level with **no file-level remedy**.

- Umbrella root — only `.github/workflows/ci.yml.disabled`.

A documented deviation is **not** an override, and neither may be recorded, reported or rationalised as one.

§5 · Gaps and risks

5.1 What does not exist and must be built

Ordered by how much of the operator’s brief depends on it.

#	Missing capability	Evidence it is missing	Consequence
1	A verbatim transcript of chapter 1	The only text asset is an 8-page, 3,044-word Gemini <i>summary</i> with no timestamps and no speaker turns (§1.3).	“Transcribe chapter 1 into markdown” is an ASR project , not a conversion. Must install an engine and choose CPU-local vs hosted, under §12.6’s 60 % RAM cap on a CPU-only host. 115 minutes of audio.
2	Any ASR tooling	<code>/usr/bin/whisper</code> is <code>whisper-1.3.1-alt1</code> by Lorenzo Paderi, not OpenAI Whisper; <code>import whisper</code> → <code>ModuleNotFoundError</code> (§3.6).	Every non-text asset must first be reduced to timestamped text. That reduction pipeline does not exist.
3	Semantic search over media	Lumen’s extension allowlist is compile-time and contains no media, image, PDF, HTML or SVG type (§3.1).	
4	Any LLM bridge	No provider registry, no chat call, no key handling,	“Ask an AI, get an instant

#	Missing capability	Evidence it is missing	Consequence
		no CA wiring in this tree; HelixLLM/HelixAgent/LLMProvider are prose only (§3.2). Nearest real code is a prompt-cache benchmark in the constitution submodule.	answer” starts from zero here.
5	A local generative model	Only two models on the host, both embedding-only, same blob; ollama is CPU-only via GGML_VK_VISIBLE_DEVICES=-1 (§3.2).	Local inference for Q&A is not currently possible without pulling a chat model.
6	Containerization of anything curriculum-shaped	ai_interviewing ships no Dockerfile/Containerfile/compose (§2.2); the only Containerfiles in the tree build an unrelated translator (§3.5).	The “runnable via containers” half has no precedent to copy, and pulls in §11.4.76 + §11.4.161 + §12.3 (§4.6).
7	The vasic-digital/containers submodule	Absent from .gitmodules; the repo exists in the org inventory (§4.6).	§11.4.76 requires it as <i>the</i> orchestration layer before any container work is compliant.
8	Ranked / autocomplete / cross-linked search	store.Search is a 4-way LIKE union, no ranking, no FTS, bodies not searched; UI has no typeahead; repo-wide grep for deepwiki\ autocomplete\ typeahead\	“deepwiki-class UX with fast autocomplete” and “navigate point-to-

#	Missing capability	Evidence it is missing	Consequence
9	A media pipeline	fts5 finds no implementation (§2.3). Ingest’s only media operation is “copy a PNG, rewrite its link” (§2.4). No transcode, no thumbnails, no waveform, no chapter markers, no subtitle track.	point” are entirely new. Audio/video playback with deep links to timestamps must be designed from scratch.
10	Multi-track / multi-chapter extensibility without code edits	Tracks are a hardcoded Go literal slice; domain grouping is a switch on ordinal (§2.1).	“Extensible by a repeatable prompt whenever a new chapter is added” is blocked until track/chapter discovery becomes data-driven.
11	Governance carriers + helix-deps.yaml entry for workshop/	workshop/ has none of the four carriers and is absent from helix-deps.yaml; G7 is 20/24 (§1.1, §4.2).	In flight by another agent — the spec should depend on it, not duplicate it.
12	Four-format exports for workshop/	workshop/README.md has no .html/.pdf siblings; G8 is repo-wide OPEN (§4.7). <i>This RECON.md inherits that gap.</i>	§11.4.65 binds every documenting .md in an owned submodule.
13	A workshop/ test/ evidence harness	self-test.sh covers only the archive pipeline; no unit/integration/e2e/ challenge banks; §11.4.169 names 13 test types and §11.4.224 sets an 85 % floor (§4.7).	Substantial, and TDD-first per §11.4.224.

5.2 Ambiguities that need a clarify question

1. **“Transcribe” — to what fidelity, by what engine, and is a hosted API acceptable?** Verbatim diarized transcript with per-utterance timestamps, or a cleaned reading transcript? Local CPU Whisper (hours, §12.6-bounded) or a hosted ASR service (sends a 1.87 GB private recording of two named individuals off-host)? **The privacy dimension is not a detail** — the recording names participants and discusses commercial strategy and pricing.
2. **Where does the curriculum live?** Inside workshop/ (self-contained, mirroring `ai_interviewing/`), or at the umbrella root with workshop/ as content-only? This determines whether the umbrella pre-push gates ever see the new code.
3. **What is a “chapter” as a data structure?** `ai_interviewing` maps one Markdown file to one module. A workshop chapter is one 2-hour recording plus a transcript plus code plus diagrams plus exercises. Is a chapter a *track* (many lessons) or a *module* (many `##` sections)?
4. **“Runnable via containers” — which surface?** The whole platform in one container? Backend + frontend + an inference sidecar? And does §11.4.76’s on-demand-infra invariant mean the bash scripts must *start* the containers, so an operator never runs podman by hand?
5. **“Lumen semantic integration” — as which of three things?** (a) the platform shells out to `lumen search` and parses plaintext (no JSON output flag exists); (b) the platform speaks MCP to `lumen stdio`; (c) the platform builds its **own** embedding index (`jina-v2`-base-code via `ollama`) and Lumen stays a developer tool. These have radically different costs, and (a) inherits the whole-repo index at `~/.local/share/lumen/21bf1507a8925bcf/`, which is scoped to the *repository*, not to the curriculum.
6. **Internal vs external LLM — what is the default, and where do keys live?** No provider registry, no key handling and no CA wiring exist here (§3.2), and the host has no chat model. §11.4.10 requires credentials be git-ignored, runtime-loaded, and separated per service.
7. **What does “instant” mean numerically?** An answer-latency budget is required before local-vs-hosted can be decided; a CPU-only host cannot serve sub-second generation.
8. **Which languages?** The umbrella renders 15; `ai_interviewing`’s platform is English-only-but-i18n-ready. A transcript is expensive to translate.

9. **Is the 1.87 GB video the distribution artefact, or is a web-optimised rendition produced?** 1080p24 at 2.16 Mb/s is not a streaming ladder, and a browsable player implies seekable delivery.
10. **What exactly is the “repeatable prompt” contract for adding chapter N?** A slash command? A skill? A script plus a prompt? Its output contract determines how much of the pipeline must be declarative.

5.3 Where the `ai_interviewing` pattern will not transfer cleanly

1. **The ingest model is Markdown-only and lossy for media.**
`ingest.go` reads `.md`, splits on `##` and `####` Q, and copies PNGs. It has no concept of a time axis. A transcript’s value is its **timestamps**, and nothing in the schema (`lesson`, `question`, `asset`) can hold one. `asset(name, mime, rel_path)` has no duration, no offset, no track.
2. **LIKE-based search does not survive a transcript.** A 115-minute transcript is roughly 15,000–18,000 words in one document. Lesson **bodies are not searched at all** today — only titles — so a LIKE search over a transcript-derived module would return the module title and nothing useful. The 40-row `LIMIT` with no ranking makes it worse.
3. **Hardcoded tracks defeat “add a chapter with a prompt.”** Three struct literals and a switch on ordinal must become data before chapter 02 can be added without a Go edit and rebuild.
4. **One flat asset namespace.** `/api/assets/:name` does `filepath.Base()` and serves from one directory (158 PNGs today). Video, audio, subtitle tracks and per-chapter media need namespacing, range requests (`Accept-Ranges`), and content-type handling that `c.File()` alone does not provide.
5. **`store.Reset()` wipes and rebuilds everything on every ingest.** Fine for a 206 MB Markdown corpus that re-ingests in ~20 s; hostile to a corpus whose derived artefacts (embeddings, transcripts, transcodes) cost hours to produce. Incremental ingest is a new requirement.
6. **`SetMaxOpenConns(1)`.** One writer keeps `modernc-sqlite` deterministic; it also serializes every read. Acceptable for a single-reader study app, a real constraint for a concurrent search + chat workload.
7. **No auth of any kind, and `AllowAllOrigins: true`.** `ai_interviewing` is a localhost study tool with a `localStorage` session id. Anything published, or anything holding a private recording of a named third party, needs a different posture.
8. **The export pipeline scales by document, not by minute.** `build-all.sh` exports every `.md` to HTML+PDF+DOCX and validates every

PDF with OCR. A 15,000-word transcript is fine; the §11.4.168 visual layer requires **≥1 embedded raster image per doc that references a diagram**, which is why `render-diagrams.sh` emits PNG rather than SVG — a constraint any new document class inherits.

9. **The SQLite macOS-path defect** (§2.5) is a live example of the portability class that `scripts/audit-hardcoded-paths.sh` catches at the umbrella root but **not inside submodules**. New workshop scripts get no such sweep for free.
10. **Three stale copies of the learning-kit** (§3.3) mean “reuse the design system” needs an explicit source-of-truth decision before any UI work starts.

5.4 Risk register

Risk	Grounding	Exposure
A workflow file added to workshop/ blocks release	§11.4.156 governs “ <i>every owned + nested submodule</i> ”; gate E only sweeps the umbrella’s git <code>ls-files</code> and would not catch it (§4.8).	High — an unguarded blind spot.
A hosted-ASR decision leaks a private recording	Named participants, commercial strategy, pricing (§1.3). No §11.4.10-shaped credential or data-egress policy exists for this.	High.
Local ASR + embedding breaches the 60 % RAM cap	§12.6 has no override; the host is CPU-only; the video is 115 min / 1.87 GB (§3.6, §4.7).	Medium-high — needs a bounded execution scope from the first task.
Lumen index churn	Changing the embedding-model <i>name</i> silently starts a second index rather than updating <code>21bf1507a8925bcf</code> (§3.1); the existing index is already 335	Medium-high.

Risk	Grounding	Exposure
Container work lands non-compliant	MB and one rebuild has taken ~15 h. §11.4.76 requires the containers submodule as the <i>sole</i> orchestration layer; every existing call site in this tree is ad-hoc and the <code>amber.local</code> branch picks Docker (§3.5, §4.6).	Medium-high.
Repository weight	<code>workshop/</code> is already 3.5 G working tree / 1.8 G <code>.git</code>. Adding transcodes, embeddings and per-chapter media multiplies it. The 50 MB part scheme is manual and hook-driven.	Medium.
<code>helix-deps.yaml</code> drift	Observed: <code>workshop</code> absent entirely, and every declared ref is stale against live <code>git submodule status</code> (constitution 448981ae→902979027, <code>design-toolkit</code> 16e4e76→efd2c3fb, <code>ai_interviewing</code> 023abbfd→ed73d855, <code>monetization</code> 1f9f5204→54ed7b0f, <code>vasic.digital</code> 5a4c3bba→6e5411c2, <code>milosvasic.ru</code> v1.8.0-5→v1.8.0-9). Reported as observation only —	Medium.

Risk	Grounding	Exposure
	another agent owns this file.	
Fresh clone is unprotected	.git/hooks/ untracked; nothing runs until --install; --no-verify bypasses silently (§4.8).	Medium.
.lumenignore may need extending	It excludes only _tests/evidence/. A workshop/ full of transcripts and generated artefacts could inflate index time toward the ~15 h figure the file already documents.	Medium.
Evidence directories are git-ignored everywhere	_tests/evidence, qa/evidence/, .test-evidence/ are all untracked. §11.4.262 demands machine evidence cited by path + hash + timestamp; untracked evidence is not reproducible by a reviewer on another clone.	Medium — needs an explicit decision.

§6 · Summary for the spec author

What exists in workshop/: one chapter, one 115-minute 1080p recording (tracked as 36 verified archive parts), one 8-page AI *summary* PDF, a genuinely well-built and self-tested video archive/extract pipeline, an upstream recipe, and a README that says “Tbd.”
No curriculum, no code, no containers, no carriers.

What ai_interviewing gives for free: the whole content→SQLite→SPA architecture; a filename-convention curriculum model with basename joins for parallel document kinds; a seven-script bash control plane

with health-gated idempotent start/stop/status and bound-address publication; curriculum versioning and i18n-readiness already in the schema; a deterministic re-runnable Markdown ingester; a four-format export pipeline with a validator-safe Mermaid→PNG step; and a three-bank, 24-challenge, 18-e2e, evidence-gated test discipline that is the house pattern for §1.1 and §11.4.169.

Top reusable components: (1) design-system/learning-kit/ — 31.7 KB of framework-free curriculum UI CSS on the --od-* token contract, already consumed by the reference SPA; (2) ai_interviewing/platform/scripts/ — the bash control-plane pattern; (3) ai_interviewing/scripts/ — build-all.sh / render-diagrams.sh / export-doc.sh / validate-exports.sh; (4) internal/ingest + internal/store as the ingest and schema starting point; (5) _tests/{export,visual}/self-validate.sh as the mutation-paired gate template; (6) scripts/lumen-reindex.sh and scripts/lumen-index-doctor.sh as the model for careful, three-valued, evidence-producing operational scripts.

The five biggest gaps or risks:

1. **There is no transcript and no ASR tooling** — chapter 1's only text is a 3,044-word summary of a 115-minute recording, and the host has no speech-to-text engine and only CPU inference.
 2. **Lumen cannot see media** — its extension allowlist is compile-time and text-only, so “semantic search across audio and video” requires a text-reduction pipeline that does not exist.
 3. **There is no LLM bridge and no local chat model** — everything named HelixLLM / HelixAgent / LLMPProvider in this tree is prose; the two local models are embedding-only.
 4. **Containers are entirely new territory and constitutionally constrained** — the reference implementation has none, §11.4.76 requires the absent vasic-digital/containers submodule as the sole orchestration layer, §11.4.161 forbids non-rootless Docker, and §11.4.156 forbids adding any CI workflow to workshop/ (a **release blocker** that gate E would not catch).
 5. **The reference implementation's search and extensibility are the two things the brief most depends on, and both are its weakest parts** — a 4-way LIKE union with no ranking that never searches lesson bodies, and hardcoded Go track literals that make “add a chapter with a prompt” impossible without a code edit.
-

§7 · Command log

Every command run during this survey. All are read-only. \$ROOT = <ext-volume-host>/Projects/vasic.

Repository shape

```
ls -la $ROOT; cat $ROOT/.gitmodules
git -C $ROOT status --short; git -C $ROOT submodule status
ls -la $ROOT/.github/workflows/ $ROOT/milosvasic.ru/.github/
    workflows/ $ROOT/vasic.digital/.github/workflows/
find $ROOT/docs -type f | sort
find $ROOT/.specify -maxdepth 3 -type f | sort
ls $ROOT/.test-evidence | wc -l ; ls -la $ROOT/.test-evidence/
    20260831T220*/
head -70 $ROOT/.specify/memory/constitution.md
head -40 $ROOT/CONTINUATION.md ; sed -n '1,80p' $ROOT/README.md
grep -n -i 'workshop' $ROOT/CONTINUATION.md $ROOT/README.md $ROOT/
    CLAUDE.md
```

workshop/

```
find $ROOT/workshop -path ./git -prune -o -print | sort
du -sh --exclude=.git $ROOT/workshop ; du -sh $ROOT/workshop/
    {chapters,docs,scripts,upstreams}
du -sh $ROOT/.git/modules/workshop ; du -cb $ROOT/workshop/
    chapters/01/*.part-* | tail -1
git -C $ROOT/workshop ls-files ; git -C $ROOT/workshop ls-files |
    wc -l
git -C $ROOT/workshop log --oneline -15 ; git -C $ROOT/workshop
    status --short
ls -la $ROOT/workshop/chapters/01/
cat $ROOT/workshop/README.md $ROOT/workshop/.gitignore $ROOT/
    workshop/.git
cat "$ROOT/workshop/chapters/01/...Recording.mp4.sha256"
cat $ROOT/workshop/scripts/{archive-videos,extract-videos,install-
    hooks,self-test}.sh
cat $ROOT/workshop/scripts/git-hooks/{post-commit,post-checkout}
    $ROOT/workshop/upstreams/GitHub.sh
cat $ROOT/workshop/docs/superpowers/specs/2026-08-28-video-
    archive-pipeline-design.md
ffprobe -i "$ROOT/workshop/chapters/01/...Recording.mp4"          #
    streams, duration
pdftinfo "$ROOT/workshop/chapters/01/...Notes by Gemini.PDF"
pdftotext -layout "...Notes by Gemini.PDF" - | head -120          #
    and | wc -lwc
```

ai_interviewing/

```
find $ROOT/ai_interviewing -path '*/node_modules' -prune -o -path
    '*/.angular' -prune \
    -o -path '*/dist' -prune -o -type f -print | sort
```

```

find $ROOT/ai_interviewing/docs -type f | sed 's/.*\./' | sort |
    uniq -c
git -C $ROOT/ai_interviewing ls-files | wc -l
ls $ROOT/ai_interviewing/platform/data/assets/*.png | wc -l
ls $ROOT/ai_interviewing/platform/content/{mcq,short}/*.json | wc
    -l
ls $ROOT/ai_interviewing/docs/interview-preparation/areas/*.md |
    wc -l
ls $ROOT/ai_interviewing/docs/{project-deep-dives,improvement-
    plans,validation-plans}/[0-9]*.md | wc -l
cat $ROOT/ai_interviewing/README.md $ROOT/ai_interviewing/
    platform/README.md
cat $ROOT/ai_interviewing/platform/scripts/
    {_common,build,start,stop,status,restart,ingest}.sh
cat $ROOT/ai_interviewing/platform/backend/{go.mod,cmd/server/
    main.go}
cat $ROOT/ai_interviewing/platform/backend/internal/{api/
    api.go,store/store.go}
head -140 $ROOT/ai_interviewing/platform/backend/internal/ingest/
    ingest.go
sed -n '140,330p' $ROOT/ai_interviewing/platform/backend/
    internal/ingest/ingest.go
cat $ROOT/ai_interviewing/platform/frontend/{package.json,src/app/
    app.routes.ts}
cat $ROOT/ai_interviewing/platform/frontend/src/app/{core/
    api.ts,features/search.component.ts}
head -60 $ROOT/ai_interviewing/platform/frontend/src/styles.scss
cat $ROOT/ai_interviewing/platform/qa/{helixqa/run-suite.sh,e2e/
    playwright.config.ts}
cat $ROOT/ai_interviewing/platform/qa/challenges/api-challenges.sh
grep -n "test(" $ROOT/ai_interviewing/platform/qa/e2e/tests/
    smoke.spec.ts
head -45 $ROOT/ai_interviewing/scripts/{build-all,render-
    diagrams,export-doc,validate-exports}.sh
cat $ROOT/ai_interviewing/
    {.gitignore,platform/.gitignore,platform/run/server.json}
curl -s --max-time 3 http://localhost:8099/api/health # → no
    response
wc -l <backend .go, frontend .ts, scripts, qa files>
sed -n '1,40p' $ROOT/ai_interviewing/docs/project-deep-dives/22-
    courses-creator.md
sed -n '1,32p' $ROOT/ai_interviewing/docs/project-deep-dives/17-
    helixllm.md

```

Cross-cutting

```

find $ROOT -path '*/node_modules' -prune -o -path '*/.git' -prune
    \
    -o \( -iname 'Dockerfile*' -o -iname '*compose*.yaml' -o
        -iname 'Containerfile*' \) -print
grep -rl -E '\b(podman|docker)\b' --include='*.sh' --
    include='*.md' --include='*.yaml' $ROOT
grep -rn -i -E 'deepwiki|autocomplete|typeahead|fts5' --
    include='*.go' --include='*.ts' $ROOT
grep -n -i 'containers' $ROOT/.gitmodules # → no
    match

```

```

for t in ffmpeg ffprobe whisper yt-dlp pandoc weasyprint mmdc
    pdftotext tesseract \
    podman docker go node npm ruby bundler python3 sqlite3
    lumen ollama; do
    printf '%-12s %s\n' "$t" "$(command -v $t || echo MISSING)";
done
head -20 /usr/bin/whisper ; rpm -qf /usr/bin/whisper
python3 -c "import whisper" # →
    ModuleNotFoundError
podman --version ; podman info --format
    '{{.Store.GraphDriverName}}
    rootless={{.Host.Security.Rootless}}'
head -60 $ROOT/_tests/GATES.md ; grep -n '^#' $ROOT/_tests/TEST-
    TYPES.md
python3 -c "..." # extract repo names + descriptions from
    _analysis/github-*.json

```

Constitution anchors (delegated; \$C = \$ROOT/submodules/constitution/Constitution.md)

```

grep -n '^### §11\.4\.156 ' $C
awk '/^### §11\.4\.156 /{f=1} f{print} f&&/^### §11\.4\.157 /
    {exit}' $C
# ...repeated for §11.4.157, §12.10, §1.1, §11.4.6, §11.4.65,
    §11.4.153, §11.4.73,
# §11.4.168, §11.4.32, §11.4.29, §11.4.36, §12.6, §9.1, §9.3,
    §11.4.262,
# §11.4.169, §11.4.224, §11.4.2, §11.4.76, §11.4.161, §12.3,
    §11.4.18,
# §11.4.67, §11.4.234, §2, §2.1
grep -n '^### §' $C | grep -i -E 'evidence|container|script'
sed -n '1,160p' $ROOT/Constitution.md # the project-local
    constitution

```

Lumen / LLM / design-system / generator (delegated; read-only, no lumen command run)

```

cat $ROOT/scripts/lumen-reindex.sh $ROOT/scripts/lumen-index-
    doctor.sh
cat $ROOT/.lumenignore $ROOT/.lumen-reindex.log
cat $ROOT/.claude/settings.json $ROOT/.claude/settings.local.json
pgrep -a lumen ; pgrep -a ollama
cat /etc/sysconfig/ollama ; ls /var/lib/ollama/.ollama/models/
    manifests/**
ls -la <home>/local/share/lumen/21bf1507a8925bcf/
# lumen 0.0.41 source: cmd/stdio.go, internal/chunker/
    languages.go,
# internal/index/index.go, internal/merkle/ignore.go, internal/
    config/{config,version}.go
find $ROOT/design-system $ROOT/design-toolkit -maxdepth 2
cat $ROOT/_tools/helixtranslate-container/
    {Containerfile,Containerfile.translator,run.sh}
sed -n '40,60p' $ROOT/_tools/helixtranslate-container.sh
cat $ROOT/_tools/gen/{main.go,go.mod,build.sh} ; ls $ROOT/_tools/
    gen/*_test.go

```



```
sed -n '160,230p;350,380p;470,490p' $ROOT/scripts/pre-push-  
    gates.sh  
cat $ROOT/helix-deps.yaml # READ ONLY – owned by  
    another agent
```

Honest boundary on this document

Per §11.4.6 and §11.4.65: this file is a .md documenting the project, so §11.4.65 requires .html + .pdf siblings kept in sync. They **do not exist** — gap **G8** is open repository-wide, and this document inherits it. The brief for this survey authorised exactly one file, so the exports were not generated. Generating them is `bash ai_interviewing/scripts/export-doc.sh docs/workshop-curriculum/RECON.md` (the script is generic — it takes any .md path), followed by the §11.4.168 validation in `validate-exports.sh`.